

Lab Worksheets – 2307421L- Ubiquitous Sensing, Computing and Communication (USCC)

Practical NO. 4: Publish–Subscribe Model — Real-Time Communication using Firebase (HTTP Protocol).

Batch: Batch 2

Group No.:

Members:

- Rushikesh Sable (202201070107)

Objective:

- To implement real-time data communication between ESP8266 and a mobile application using Firebase as a publish–subscribe platform.
- To publish live sensor readings (temperature, humidity, moisture, and pH) from ESP8266 to Firebase.
- To subscribe to database updates from the Bolt.AI app for real-time display.
- To verify end-to-end live data exchange between IoT device and mobile app.

Tools & Technology Used:

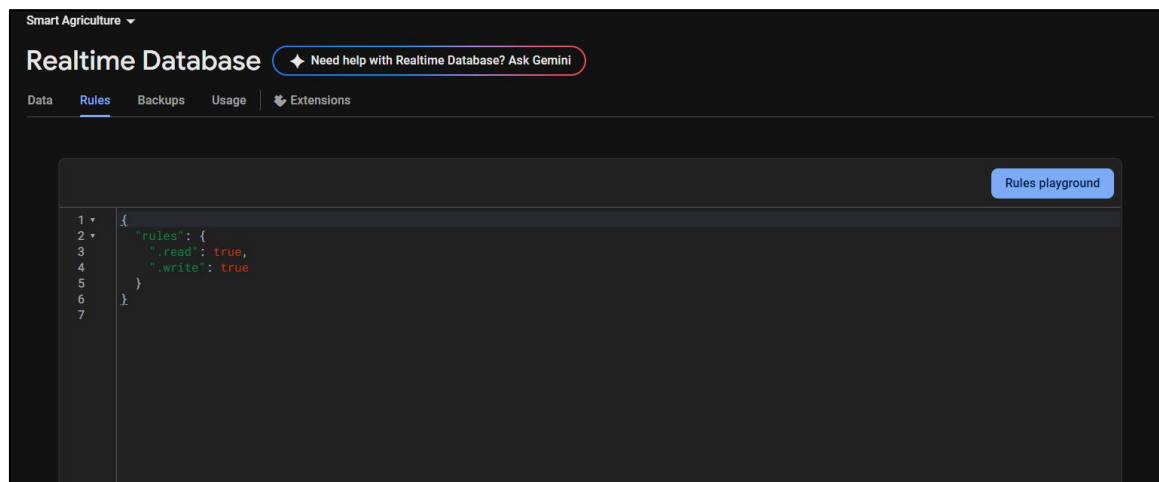
Tool / Component	Purpose
ESP8266 (NodeMCU)	Microcontroller for reading sensor data
DHT11 Sensor	Measures temperature and humidity
Soil Moisture Sensor	Detects soil moisture level
pH Sensor	Measures soil acidity / alkalinity
Firebase Realtime Database	Acts as cloud publish–subscribe channel

Tool / Component	Purpose
Bolt.AI App	Subscribes to Firebase updates and displays data
Wi-Fi Network	Enables internet communication between ESP8266 and Firebase

Procedure:

1. Firebase Setup:-

- Created a new Firebase project and enabled Realtime Database.
- Configured rules to test mode: `{"read": true, "write": true}`.
- Created database nodes for "Temperature", "Humidity", "Moisture", and "pH".



2. Publisher Setup (ESP8266):-

- Connected DHT11, Soil Moisture, and pH sensors to ESP8266.
- Programmed ESP8266 to read sensors and upload values to Firebase every 10 seconds.
- Verified successful data publishing via Serial Monitor and Firebase console.

3. Subscriber Setup (Bolt.AI App):-

- Linked Bolt.AI app with Firebase project using database URL.
- Configured app widgets to subscribe to nodes /Temperature, /Humidity, /Moisture, and

/pH.

- Set up real-time labels and charts for data visualization.

4. Testing and Validation:-

- Observed that when ESP8266 pushed new values, they instantly appeared on the app.
- Simulated sensor changes (e.g., moisture decrease) and verified app updated within seconds.
- Confirmed multiple devices could view live data simultaneously.

Code Snippets / Configuration:

```
#include <ESP8266WiFi.h>
#include <Firebase_ESP_Client.h>
#include <DHT.h>

// ----- WiFi & Firebase Credentials -----
#define WIFI_SSID "Airtel_Mighty Raju"
#define WIFI_PASSWORD "Mitaoe#701"

#define API_KEY "AIzaSyAUxLwgUALpPcWpp7_5GwHyNHsgNJQ0308" // your full key
#define DATABASE_URL "https://smart-agriculture-df146-default-rtdb.firebaseio.com/"

// ----- Sensor Pins -----
#define DHTPIN D4
#define DHTTYPE DHT11
#define ANALOG_PIN A0

DHT dht(DHTPIN, DHTTYPE);

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

// ----- Setup -----
void setup() {
  Serial.begin(115200);
  delay(1000);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
```

```

Serial.println("\n✔WiFi connected!");

// Firebase configuration
config.api_key = API_KEY;
config.database_url = DATABASE_URL;

// Anonymous sign-in
if (Firebase.signUp(&config, &auth, "", "")) {
    Serial.println("✔Firebase sign-up successful!");
} else {
    Serial.printf("✗Sign-up Error: %s\n", config.signer.signupError.message.c_str());
}

Firebase.begin(&config, &auth);
Firebase.reconnectWiFi(true);

dht.begin();
Serial.println("✔Firebase initialized!");
}

// ----- Sensor Functions -----
float readPH() {
    int samples = 10;
    float total = 0;
    for (int i = 0; i < samples; i++) {
        total += analogRead(ANALOG_PIN);
        delay(10);
    }
    float avgADC = total / samples;
    float voltage = avgADC * (3.3 / 1023.0);
    float pHValue = 7 + ((2.5 - voltage) / 0.18);
    return pHValue;
}

int readMoisture() {
    int val = analogRead(ANALOG_PIN);
    return map(val, 1023, 0, 0, 100);
}

// ----- Main Loop -----
void loop() {
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();
    int moisture = readMoisture();
    float pH = readPH();

    Serial.println("\n  Sensor Readings:");
    Serial.printf("Temperature: %.2f °C\n", temperature);

```

```

Serial.printf("Humidity: %.2f%%\n", humidity);
Serial.printf("Moisture: %d%%\n", moisture);
Serial.printf("pH Value: %.2f\n", pH);

if (Firebase.ready()) {
  // Create unique path using timestamp
  String path = "/SmartFarm/Readings/" + String(millis());

  Firebase.RTDB.setFloat(&fbdo, path + "/SmartAgri/Temperature", temperature);
  Firebase.RTDB.setFloat(&fbdo, path + "/SmartAgri/Humidity", humidity);
  Firebase.RTDB.setInt(&fbdo, path + "/SmartAgri/Moisture", moisture);
  Firebase.RTDB.setFloat(&fbdo, path + "/SmartAgri/pH", pH);

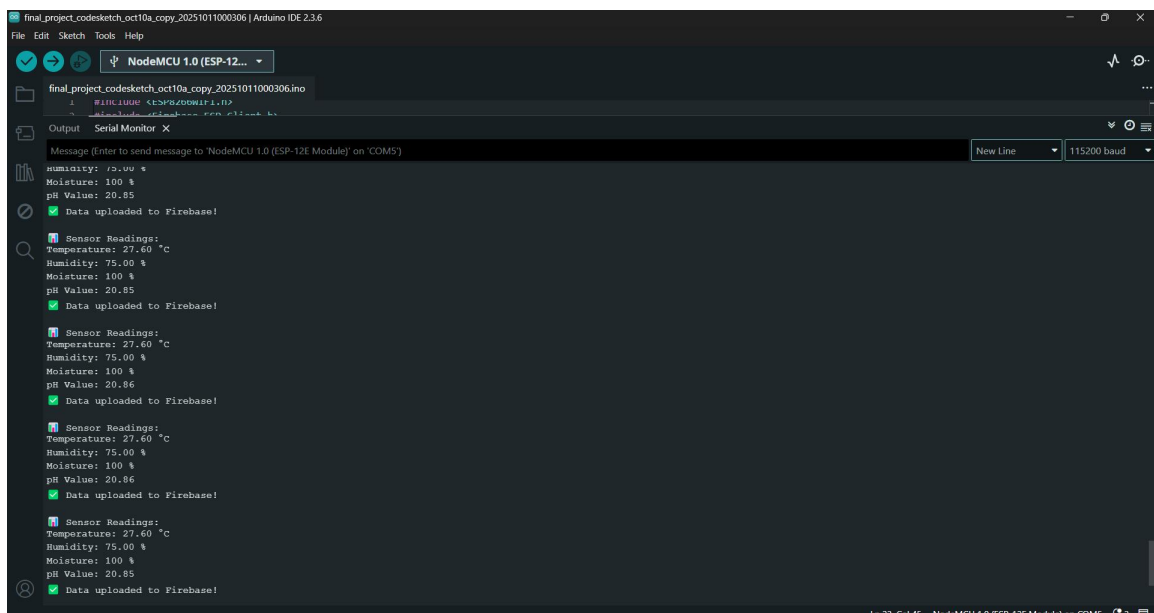
  Serial.println("✔ Data uploaded to Firebase!");
}

delay(10000);
}

```

Observations / Result:

- *Serial Monitor Output:-*



The screenshot shows the Arduino IDE interface with the Serial Monitor open. The monitor displays the following output:

```

humidity: 75.00 %
Moisture: 100 %
pH Value: 20.85
✔ Data uploaded to Firebase!

Sensor Readings:
Temperature: 27.60 °C
Humidity: 75.00 %
Moisture: 100 %
pH Value: 20.85
✔ Data uploaded to Firebase!

Sensor Readings:
Temperature: 27.60 °C
Humidity: 75.00 %
Moisture: 100 %
pH Value: 20.86
✔ Data uploaded to Firebase!

Sensor Readings:
Temperature: 27.60 °C
Humidity: 75.00 %
Moisture: 100 %
pH Value: 20.86
✔ Data uploaded to Firebase!

Sensor Readings:
Temperature: 27.60 °C
Humidity: 75.00 %
Moisture: 100 %
pH Value: 20.85
✔ Data uploaded to Firebase!

```

The Serial Monitor is connected to a NodeMCU 1.0 (ESP-12E Module) on COM5 at 115200 baud. The output shows periodic sensor readings and successful Firebase uploads.

Fig : ESP8266 Serial Monitor Output showing sensor values.

- *Cloud Data Logs:-*

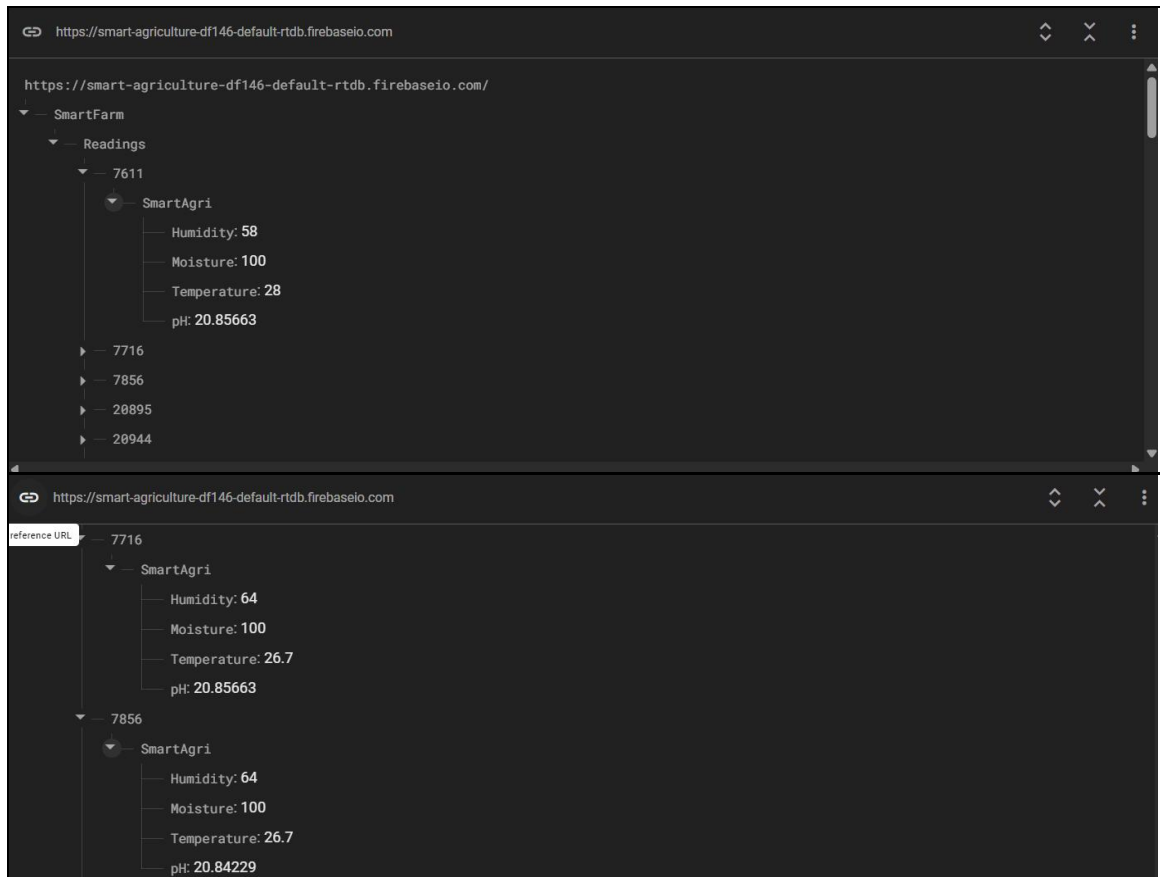




Fig 2: Firebase Realtime Database with updated JSON data.

- *Mobile App UI:-*

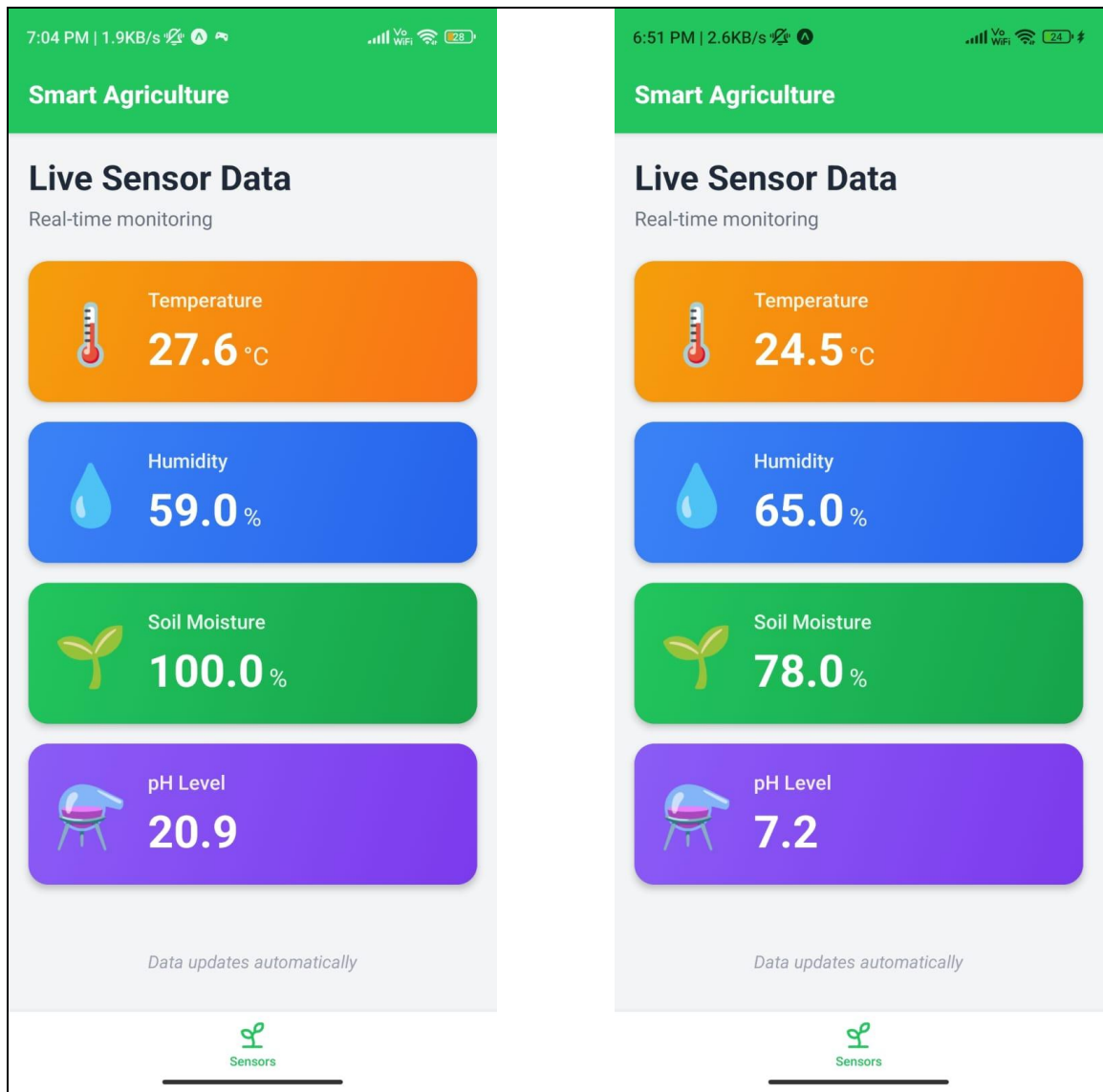


Fig:- Bolt.AI App UI displaying live values and graphs.

Troubleshooting & Debugging:

1. No Data Appearing in Firebase Console

- Problem: The ESP8266 successfully connected to Wi-Fi but Firebase showed no data.
- Cause: Incorrect Firebase host URL or invalid authentication key.
- Solution: Verified Firebase host (must end with .firebaseio.com) and rechecked database secret key under Service Accounts → Database Secrets.

2. App Not Updating in Real Time

- Problem: The Bolt.AI app displayed old data and required manual refresh.
- Cause: Static data fetch configuration instead of real-time listener.
- Solution: Configured the app widgets to use Firebase live sync, enabling instant UI updates whenever data changes.

3. Permission Denied Error

- Problem: ESP8266 failed to write to the Firebase database.
- Cause: Firebase security rules prevented external writes.
- Solution: Temporarily set read/write rules to true during development, later re-enabled authentication for security.

4. Fluctuating pH Sensor Values

- Problem: pH readings changed abruptly due to signal noise.
- Solution: Added capacitor-based filtering on the analog input and averaged multiple samples before uploading to Firebase.

5. Slow Data Response on App

- Problem: App took time to reflect changes.
- Cause: Network latency due to weak Wi-Fi.
- Solution: Switched to a 2.4 GHz Wi-Fi band and reduced data upload frequency to 2 seconds.

Conclusion:

In this practical, we successfully implemented real-time publish–subscribe communication between ESP8266 and the Bolt.AI mobile application using Firebase Realtime Database.

The ESP8266 acted as the publisher, continuously pushing sensor data to the cloud, while the mobile app acted as the subscriber, instantly updating its interface whenever the database changed.

We learned:

- How Firebase can be used as a real-time pub-sub system for IoT data transfer.
- The importance of cloud synchronization and live event listeners in data-driven applications.
- How to handle cloud communication errors and network delays effectively.
- How to ensure stability and reliability in real-time IoT applications.

This experiment demonstrates that even without MQTT, Firebase’s HTTP-based Realtime Database can efficiently replicate publish–subscribe behavior.

Such systems form the core of Smart Agriculture, Home Automation, and Environmental Monitoring platforms, allowing instant cloud-to-app communication.